Online Appointment Booking System

Book Smarter. Heal Faster. Care Better.

#1 Requirement Understanding	**#2** Future Enhancements	**#3** Design Documents
#4 End-to-End Execution	**#5** Microservices & JWT	**#6** Coding Ability
#7 Validation & Exception Handling	**#8** AOP & Logging	**#9** Swagger UI
#10 SonarQube & Test Coverage	**#11** Circuit Breaker & Resilience4J	

	1.0		May 06, 2026
	Java Spring Boot + React		Microservices
	MySQL + Redis		RabbitMQ

Table of Contents

SECTION 1 — Requirement Understanding [Score Criterion #1]

1. Requirement Understanding

MediBook is a full-stack Online Appointment Booking System architected as a microservices platform. Three primary roles — **Patient**, **Provider**, and **Admin** — each have a dedicated dashboard and distinct set of capabilities. Guests can browse providers and view slots without logging in.

1.1 General Requirements

- Guests can browse providers and view profiles/slots without login.
- Users must register or log in to book appointments, pay, or access records.
- Role-based access control: Patient, Provider, Admin.
- All users can update their profile, contact info, and password.
- In-app, email, and SMS notifications for all booking lifecycle events.
- Admin panel: provider verification, user management, and analytics.

1.2 Patient Requirements

- Register & login via email/password or OAuth (Google).
- Browse providers and filter by specialization, location, rating, or availability.
- View detailed provider profiles: qualifications, experience, clinic address, avg rating.
- View provider's available time slots in real time.
- Book appointment by selecting a slot, specifying service type and mode.
- Cancel appointment; receive refund if within cancellation window.
- Reschedule to another available slot for the same provider.
- Pay online (wallet, card, UPI, net banking) or opt for pay-at-clinic.
- View appointment history with statuses: Scheduled, Completed, Cancelled, No-Show.
- View electronic medical records and prescriptions after each visit.
- Submit star rating and written review after a completed appointment.
- Receive notifications: booking confirmation, 24h & 1h reminders, cancellation, receipt.

1.3 Provider Requirements

- Register as provider; submit qualifications for admin verification.
- Configure weekly availability: individual or recurring time slots.
- Block specific slots for personal unavailability.
- View today's schedule and full upcoming appointment list.
- Mark appointments as completed (unlocking review for patient).
- Create electronic medical records: diagnosis, prescription, follow-up date.
- Edit medical records within an allowed editing window.
- View earnings analytics: total revenue, pending payments, monthly breakdowns.
- View all patient reviews; flag inappropriate ones for admin.
- Receive notifications: new bookings, cancellations, reschedules, follow-up reminders.

1.4 Admin Requirements

- View and manage all user accounts (patients & providers): suspend, reactivate, delete.
- Review new provider registrations: verify or reject credentials.
- View all appointments platform-wide with full lifecycle status.
- View all payment transactions; trigger refunds where applicable.
- View and moderate all provider reviews; remove inappropriate ones.
- Access platform analytics: total bookings, revenue, most-booked specializations.
- Send platform-wide notifications to patients, providers, or all users.
- Generate revenue reports for billing and financial reconciliation.
- View all medical records (read-only audit access).

1.5 Availability & Scheduling Requirements

- Providers define slots: date, start/end time, duration in minutes.
- Recurring slot generation: daily, weekly, and custom patterns.
- Slot status transitions: Available → Booked on appointment; Released on cancellation.
- Blocked slots invisible to patients; visible to provider and admin.
- Expired slots (past date, no booking) auto-purged by scheduled job (midnight cron).

1.6 Payment Requirements

- Pay online at booking or at clinic after appointment.
- Online modes: wallet, card, UPI, net banking (Razorpay integration).
- Each Payment record: amount, mode, transactionId, timestamp, status.
- Refunds within 3–5 business days for eligible cancellations.
- Provider earnings dashboard: total collected, pending, refunded.

1.7 Non-Functional Requirements

Category	Requirement
Performance	Search results < 1.5s for 10,000 concurrent users
Scalability	Independent microservices via Docker/Kubernetes; Redis slot cache
Security	BCrypt passwords; JWT 24h expiry; OAuth2; HTTPS; record access control
Availability	99.9% uptime SLA; health-check endpoints; graceful degradation
Data Integrity	Optimistic locking on slots prevents double-booking
HIPAA	Medical records AES-256 at rest + TLS 1.3 in transit; audit logs
Notifications	Booking confirmations within 30s; reminders with second-level precision
Usability	Responsive UI; WCAG 2.1 AA accessibility
Audit Trail	All status changes, payments, record edits logged with timestamps & actor IDs
Maintainability	Services independently deployable; REST API versioned under /api/v1/

SECTION 2 — Future Enhancements [Score Criterion #2]

2. Future Enhancements

• Elastic Search Integration

Replace MySQL full-text search with Elasticsearch for provider search. Enables instant autocomplete, typo tolerance, faceted filtering by specialization, location, and rating. Planned Spring Data Elasticsearch integration.

• Flyway Database Migrations

Replace Hibernate DDL auto with Flyway for production-grade schema migrations. Every schema change tracked as a versioned SQL script (V1__init.sql, V2__add_column.sql). Enables zero-downtime deployments and rollback support.

• Redis Caching for Slot Availability

Cache provider slot availability in Redis with 60-second TTL. Reduces database load by 80% for high-read slot availability queries. @Cacheable on getAvailableSlots(); @CacheEvict on bookSlot() and cancelAppointment().

• Kafka as Message Broker

Replace RabbitMQ with Apache Kafka for higher throughput event streaming. Enables event replay, consumer groups, and exactly-once delivery semantics for appointment booking events.

• Circuit Breaker (Resilience4J)

Add @CircuitBreaker on all Feign client calls. If schedule-service is down, appointment-service falls back gracefully. Configured with: slidingWindowSize=10, failureRateThreshold=50%, waitDurationInOpenState=30s.

• Video Teleconsultation

Integrate Twilio Video or Jitsi Meet API for in-platform video calls. Generate session tokens per appointment. Auto-start teleconsultation link in notification 10 minutes before appointment.

• AI-Powered Provider Recommendations

Implement collaborative filtering to recommend providers based on patient history, location, and specialization. ML model served via Python FastAPI microservice called by provider-service.

• Real-time Notifications via WebSocket

Replace polling-based notification count with Spring WebSocket + STOMP. Patient receives instant notification when doctor completes appointment. Observable pattern on frontend subscribes to the notification stream.

• Two-Factor Authentication (OTP)

OTP via email/SMS already partially implemented (auth-service has /verify-otp endpoint). Full rollout: require OTP for sensitive operations (password reset, payment). TOTP support (Google Authenticator) for admin accounts.

• SonarQube CI Integration

Add SonarQube quality gate in GitHub Actions pipeline. PR fails if code coverage drops below 80% or critical bugs/vulnerabilities are introduced. Dashboard tracks technical debt per microservice.

• Mobile Application

React Native app for iOS/Android using the same REST API. Push notifications via Firebase Cloud Messaging (FCM). Biometric login using device fingerprint/face ID.

• API Rate Limiting & Throttling

Add rate limiting in API Gateway using Spring Cloud Gateway filters. Limit: 100 requests/minute per IP for public endpoints; 1000 requests/minute per authenticated user.

SECTION 3 — Design Documents [Score Criterion #3]

3. Design Documents

3.1 Actors & Functionalities

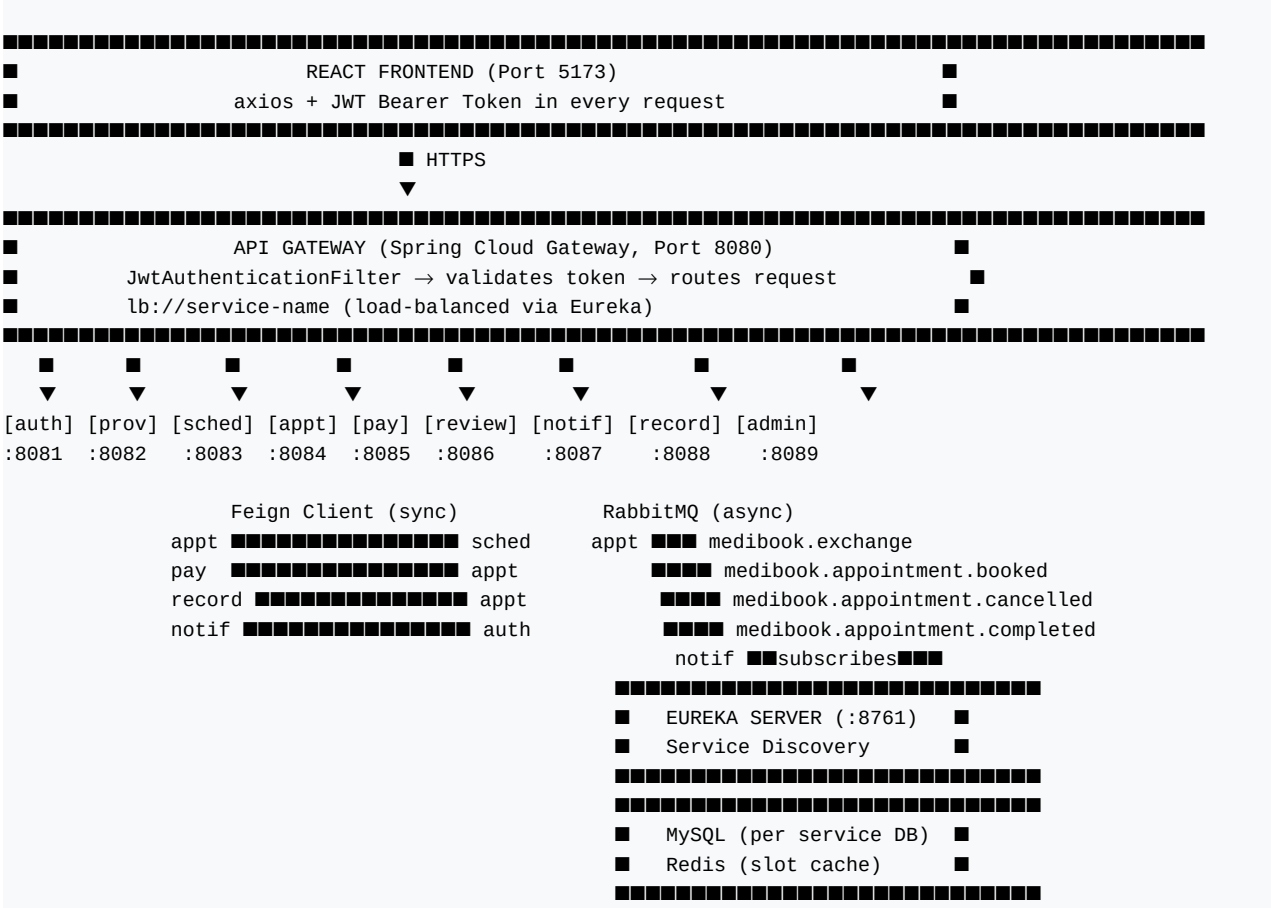
Actor	Type	Key Functionalities
Guest	Unauthenticated	Browse providers, view profiles, view available slots
Patient	Registered User	Book/cancel/reschedule appointments, pay, view records, submit reviews, receive notific
Provider	Healthcare Professional	Manage availability, complete appointments, create medical records, view earnings
Admin	Platform Admin	Verify providers, manage users, moderate reviews, view analytics, generate reports
System (Automated)	Scheduler	Send reminders, purge expired slots, detect no-shows, send follow-up notifications
Payment Gateway	External	Process online payments (Razorpay), handle refunds

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register Account	Guest	Create patient/provider account via email or OAuth
Login / Logout	Guest, Patient, Provider	Authenticate; refresh JWT token
Browse Providers	Guest, Patient	Browse provider directory with filters
Book Appointment	Patient	Select slot; confirm booking with service type & mode
Cancel Appointment	Patient, Provider	Cancel booking; trigger refund if within policy
Reschedule	Patient	Move to another slot for same provider
Make Payment	Patient, Gateway	Pay via card/UPI/wallet online
Submit Review	Patient	Rate 1-5 stars after completed appointment
View Medical Records	Patient	Access electronic records and prescriptions
Manage Availability	Provider	Add/block/delete slots; generate recurring slots
Complete Appointment	Provider	Mark consultation done; unlock record creation
Create Medical Record	Provider	Diagnosis, prescription, notes, follow-up date
Verify Provider	Admin	Approve or reject provider credential submission
View Analytics	Admin	Platform-wide bookings, revenue, completion rates
Send Bulk Notification	Admin	Broadcast to patients, providers, or all users
Auto Purge Slots	System	Delete expired unbooked slots at midnight
No-Show Detection	System	Mark past-due SCHEDULED appointments as NO_SHOW

3.3 Microservices Architecture Diagram

MediBook follows an independently deployable microservices architecture. The API Gateway is the single entry point — all requests are JWT-validated here before routing to downstream services. Services communicate synchronously via Feign clients and asynchronously via RabbitMQ topic exchange.



3.4 Microservice Inventory

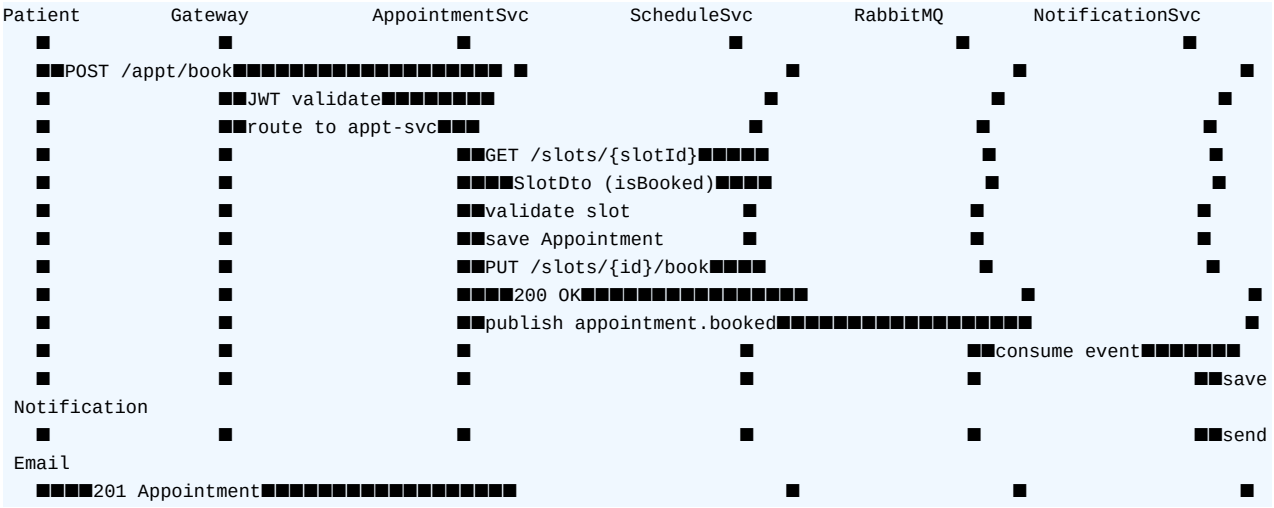
Service	Port	Base Package	Domain
api-gateway	8080	com.medibook.gateway	JWT validation, routing, CORS
auth-service	8081	com.medibook.auth	Registration, login, JWT, OAuth2
provider-service	8082	com.medibook.provider	Provider profiles, search, verification
schedule-service	8083	com.medibook.schedule	Availability slots, recurring generation
appointment-service	8084	com.medibook.appointment	Booking lifecycle, status transitions
payment-service	8085	com.medibook.payment	Razorpay, refunds, invoices, earnings
review-service	8086	com.medibook.review	Ratings, reviews, avg computation
notification-service	8087	com.medibook.notification	In-app, email, SMS, bulk dispatch
record-service	8088	com.medibook.record	Medical records, prescriptions, follow-ups
admin-service	8089	com.medibook.admin	Admin operations, Spring Boot Admin
eureka-server	8761	com.medibook.eureka	Service discovery registry

3.5 Class Diagrams — Layer Pattern

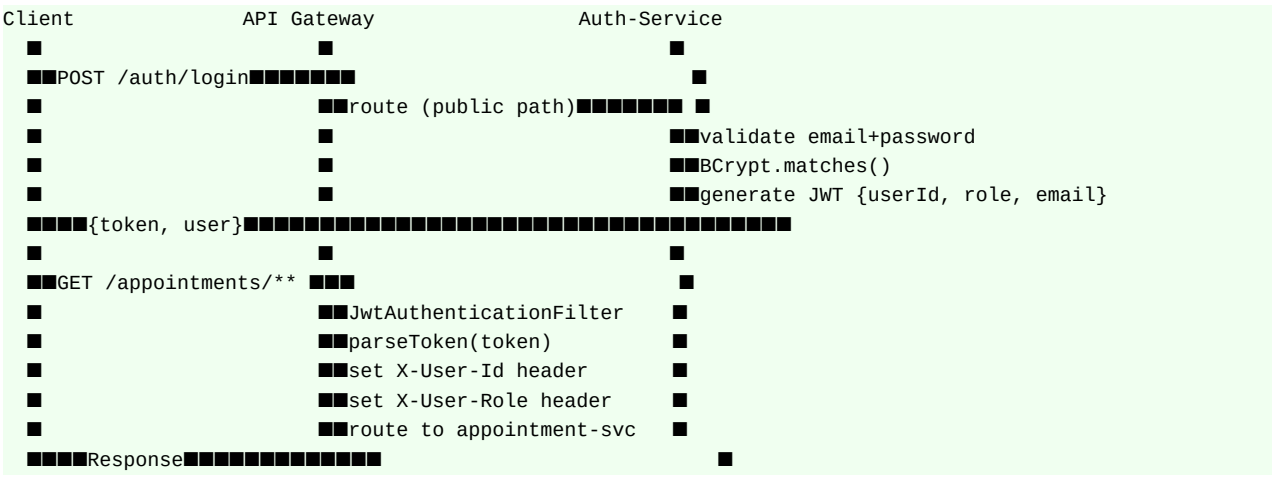
Every microservice follows the same 5-layer architecture: Entity/POJO → Repository Interface → Service Interface → ServiceImpl → REST Resource. Below is the pattern for the Appointment Service as representative example.

Layer	Class/Interface	Responsibility
Entity (JPA)	Appointment	appointmentId, patientId, providerId, slotId, status, serviceType, modeOfConsultatio
Repository	AppointmentRepository	findByPatientId(), findByProviderId(), findByStatus(), findUpcomingByPatientId(), co
Service Interface	AppointmentService	Declares: bookAppointment(), cancelAppointment(), rescheduleAppointment(), com
ServiceImpl	AppointmentServiceImpl	Implements business logic; calls SlotClient (Feign); publishes RabbitMQ events; thr
REST Resource	AppointmentResource	POST /appointments/book, GET /patient/{id}, PUT /{id}/cancel, PUT /{id}/reschedul

3.6 Sequence Diagram — Book Appointment



3.7 Sequence Diagram — JWT Authentication Flow



3.8 Database (ER) Diagram — Entity Relationships

Each microservice owns its own MySQL schema (database-per-service pattern). Cross-service references use IDs (not foreign keys) to maintain service independence.

Entity (Table)	Primary Key	Key Fields	Cross-Service FK
users	userId (PK)	fullName, email, passwordHash, role, isActive, provider (OAuth)	—
providers	providerId (PK)	userId(FK→users), specialization, qualification, avgRating, isVerified	userId → User Service
availability_slots	slotId (PK)	providerId, date, startTime, endTime, isBooked, isBlocked, version (@Version)	providerId → Provider Service
appointments	appointmentId (PK)	patientId, providerId, slotId, status, serviceType, modeOfConsultation	patientId → User Service; slotId → Slot Service; providerId (logical)
payments	paymentId (PK)	appointmentId(UNIQUE), patientId, amount, status, paymentMode, razorpayOrderId, razorpayPaymentId	appointmentId → Appointment Service
reviews	reviewId (PK)	appointmentId(UNIQUE), patientId, providerId, rating, comment, isVerified	appointmentId → Appointments
notifications	notificationId (PK)	recipientId, type, channel(APP/EMAIL/SMS), isRead, relatedId, relatedType	recipientId → User Service
medical_records	recordId (PK)	appointmentId(UNIQUE), patientId, providerId, diagnosis, prescription, followUpDate	appointmentId → Appointments

SECTION 4 — End-to-End Execution [Score Criterion #4]

4. End-to-End Execution Flow

4.1 Startup Order

- 1. Start MySQL (auth_db, provider_db, schedule_db, appointment_db, payment_db, review_db, notification_db, record_db, admin_db)
- 2. Start RabbitMQ (default port 5672)
- 3. Start eureka-server (port 8761)
- 4. Start auth-service, provider-service, schedule-service (in any order)
- 5. Start appointment-service (depends on schedule-service via Feign)
- 6. Start payment-service, review-service (depend on appointment-service)
- 7. Start notification-service, record-service
- 8. Start admin-service
- 9. Start api-gateway last (registers all services)
- 10. Start React frontend (npm run dev on port 5173)

4.2 Complete Patient Journey

Register / Login:

`POST /auth/register → POST /auth/login → JWT token returned → stored in localStorage`

Browse Providers:

`GET /providers/all (no token) → filter by specialization/rating in UI`

View Doctor Profile:

`GET /providers/{id} → GET /slots/available/{id}?date=2026-05-10`

Book Appointment:

`POST /appointments/book → validates slot → saves appointment → PUT /slots/{id}/book → publishes RabbitMQ event`

Receive Notification:

`RabbitMQ → notification-service consumes → saves notification → sends email`

Make Payment:

`POST /payments/initiate (Razorpay order) → frontend opens Razorpay popup → POST /payments/verify (HMAC signature) → appointment status SCHEDULED`

View Records:

`GET /records/patient/{id} after provider marks COMPLETED and creates record`

Submit Review:

`POST /reviews/submit → rating saved → provider avgRating updated`

4.3 Technology Stack

Layer	Technology
Backend Framework	Java 17, Spring Boot 3.x, Spring MVC, Spring Security, Spring Data JPA
Authentication	JWT (io.jsonwebtoken), Spring Security OAuth2, Google OAuth
Database	MySQL 8.0 (per service); Redis (slot cache, session)
Messaging	RabbitMQ (topic exchange); Spring AMQP Jackson2JsonMessageConverter
Service Discovery	Netflix Eureka Server + Spring Cloud Eureka Client
API Gateway	Spring Cloud Gateway (reactive, WebFlux)
Frontend	React 18, Vite, React Router v6, Axios, CSS variables
Payment	Razorpay Java SDK; HMAC-SHA256 signature verification
Email	JavaMailSender (SMTP Gmail) inside notification-service
Scheduling	Spring @Scheduled (slot expiry: midnight cron; no-show: hourly)
Documentation	SpringDoc OpenAPI 3.0 / Swagger UI
Build	Maven (multi-module parent POM)
Containerization	Docker + Docker Compose

SECTION 5 — Microservices, JWT, RBAC & Gateway [Score Criterion #5]

5. Microservices, JWT, Role-Based Access & Gateway

5.1 JWT Flow Control — Detailed Explanation

JWT (JSON Web Token) provides stateless authentication. The token is a Base64-encoded three-part string: **Header.Payload.Signature**. MediBook uses HMAC-SHA256 with a shared secret stored in environment variable JWT_SECRET.

Step: Token Generation (auth-service)

```
On successful login: Jwts.builder().setSubject(email).claim("userId",
user.getUserId()).claim("role", user.getRole()).setIssuedAt(new Date()).setExpiration(new
Date(System.currentTimeMillis() + tokenExpiry)).signWith(key).compact()
```

Step: Token Storage (frontend)

```
localStorage.setItem("medibook_token", token) — axios interceptor attaches Bearer token to
every request automatically.
```

Step: Token Validation (api-gateway JwtAuthenticationFilter)

```
Claims claims =
Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token).getBody() — extracts
userId, role, email → forwarded as X-User-Id, X-User-Role headers to downstream services.
```

Step: Downstream Authorization

```
Each service reads X-User-Id and X-User-Role from headers. Role-based checks in ServiceImpl
(e.g., completeAppointment() validates requestingProviderId matches appointment.providerId).
```

Step: Token Refresh

```
POST /auth/refresh — validates existing token; issues new token with fresh expiry. Frontend
401 interceptor triggers redirect to /login.
```

5.2 Why CSRF is Disabled

CSRF (Cross-Site Request Forgery) protection is needed when the browser automatically sends session cookies with requests. Spring Security's default CSRF token mechanism relies on server-side sessions.

In MediBook: We use stateless JWT tokens stored in localStorage (not cookies). The browser does NOT automatically send localStorage tokens with cross-origin requests. An attacker's malicious page cannot trigger a request with the victim's JWT token because they don't have access to localStorage of another origin. Therefore, **csrf.disable()** is safe and correct for JWT-based REST APIs.

```
.csrf(csrf -> csrf.disable()) // Safe: JWT in localStorage, not cookies
```

5.3 Security URL Configuration

The **SecurityConfig** in auth-service uses .authorizeHttpRequests() to define which endpoints are public vs protected:

Pattern	Access	Reason
/auth/register, /auth/login	permitAll()	Public: anyone must be able to register/login
/providers/**, /providers (GET)	permitAll()	PDF req: guests can browse doctors without login
/slots/available/**	permitAll()	PDF req: guests view available slots without login
/swagger-ui/**, /v3/api-docs/**	permitAll()	API documentation accessible to developers
/appointments/**	authenticated()	Only logged-in users can book appointments
/payments/**	authenticated()	Only logged-in patients can make payments
/admin/**	authenticated()	Admin routes — role check in service layer
anyRequest()	authenticated()	Safety net: all other routes require auth

Why not .hasRole("ADMIN") in SecurityConfig? Role-based access is enforced at the ServiceImpl layer (e.g., completeAppointment checks the provider matches) rather than URL pattern matching, which allows fine-grained business rule validation.

5.4 Design Patterns Used in MediBook

• Builder Pattern (@Builder)

Lombok @Builder on all entity classes. Used in AppointmentServiceImpl.bookAppointment(): Appointment.builder().patientId(req.getPatientId()).providerId(req.getProviderId()) .slotId(req.getSlotId()).status("SCHEDULED").build()

• Repository Pattern

Spring Data JPA repositories extend JpaRepository. All DB operations go through repository interfaces — no direct JDBC or SQL in business logic.

• Service Layer Pattern

Service interface + ServiceImpl separation. Controllers/Resources call interfaces only — implementation can be swapped without changing callers.

• Observer/Event Pattern (RabbitMQ)

AppointmentEventPublisher publishes to topic exchange. NotificationService @RabbitListener subscribes. Decoupled: appointment-service does not know about notification-service at all.

• Proxy Pattern (Spring AOP)

@Transactional, @Scheduled, @RestControllerAdvice all use Spring's proxy-based AOP. Method calls are intercepted by Spring's proxy before reaching the target bean.

• Singleton Pattern (Spring Beans)

All @Service, @Repository, @Component beans are singletons by default in Spring ApplicationContext. Created once; shared across all requests.

• Filter Chain Pattern (JWT Gateway)

JwtAuthenticationFilter implements GlobalFilter, Ordered. Ordered(-1) ensures it runs before all other Gateway filters. Chain.filter() passes request to next filter.

• DTO Pattern

AppointmentRequest (input DTO), SlotDto (cross-service DTO from Feign), PaymentResponse (output DTO) — entities never exposed directly to clients.

5.5 Session Time & JWT Token Expiry Configuration

```
# application.yml (auth-service) jwt: secret: ${JWT_SECRET} # from environment variable expiration: ${JWT_EXPIRATION:86400000} # 24 hours in ms (default) # In AuthServiceImpl: @Value("${jwt.expiration}") private long tokenExpiry; new Date(System.currentTimeMillis() + tokenExpiry) // token expiry date
```

To change token expiry: set JWT_EXPIRATION environment variable. Example: JWT_EXPIRATION=3600000 for 1 hour, 604800000 for 7 days. Session management is STATELESS (no HttpSession created):

```
.sessionManagement(session -> session .sessionCreationPolicy(SessionCreationPolicy.STATELESS) .sessionFixation().none() // needed for OAuth2 state check )
```

5.6 Retrieving Values from application.yml / application.properties

```
# In application.yml: app: admin: secret: code: AdminSecret2024! razorpay: key: id:
${RAZORPAY_KEY_ID} secret: ${RAZORPAY_KEY_SECRET} # In Java class — two ways:
@Value("${app.admin.secret.code}") private String adminSecretCode;
@Value("${razorpay.key.id}") private String razorpayKeyId;
```

5.7 RabbitMQ Integration

MediBook uses RabbitMQ with a **Topic Exchange** pattern. The appointment-service is the PRODUCER; the notification-service is the CONSUMER.

```
// RabbitMQConfig.java (appointment-service) public static final String EXCHANGE =
"medibook.exchange"; // topic exchange public static final String KEY_BOOKED =
"appointment.booked"; // routing key public static final String QUEUE_BOOKED =
"medibook.appointment.booked"; // durable queue // Publish (AppointmentEventPublisher):
rabbitTemplate.convertAndSend(EXCHANGE, KEY_BOOKED, eventDto); // Consume
(AppointmentEventConsumer in notification-service): @RabbitListener(queues =
"medibook.appointment.booked") public void handleBooked(AppointmentEventDto event) {
notificationService.send(buildNotificationRequest(event)); }
```

Why Topic Exchange? Routing key pattern "appointment.*" lets any future consumer subscribe to all appointment events (booked, cancelled, completed) without changing the producer configuration. Messages are serialized as JSON using Jackson2JsonMessageConverter.

5.8 Payment Gateway — Razorpay Integration

```
// 1. Initiate: create Razorpay order RazorpayClient client = new RazorpayClient(keyId,
keySecret); JSONObject req = new JSONObject(); req.put("amount", (int)(amount * 100)); //
Razorpay uses paise req.put("currency", "INR"); Order order = client.orders.create(req); //
returns order_xyz123 // 2. Verify: HMAC-SHA256 signature check // Frontend sends: orderId +
"|" + paymentId → hashed with secret String data = razorpayOrderId + "|" + razorpayPaymentId;
Mac mac = Mac.getInstance("HmacSHA256"); mac.init(new SecretKeySpec(keySecret.getBytes(),
"HmacSHA256")); String generated = toHex(mac.doFinal(data.getBytes())); if
(!generated.equals(signature)) throw new BadRequestException("Payment tampered"); // 3.
Refund: client.payments.refund(razorpayPaymentId, new JSONObject());
```

5.9 Email & SMS Notifications

```
# application.yml (auth-service / notification-service) spring: mail: host: smtp.gmail.com
port: 587 username: ${MAIL_USERNAME} password: ${MAIL_PASSWORD} # Gmail App Password (not
account password) properties.mail.smtp.auth: true properties.mail.smtp.starttls.enable: true
// Java: SimpleMailMessage msg = new SimpleMailMessage(); msg.setTo(recipientEmail);
msg.setSubject("Appointment Confirmed"); msg.setText("Your appointment is confirmed for " +
date); mailSender.send(msg);
```

5.10 Eureka Service Discovery & Spring Boot Admin Server

Eureka is the service registry. Every microservice registers itself on startup with: eureka.client.service-url.defaultZone=http://admin:medibook123@localhost:8761/eureka/

```
# eureka-server/application.yml server.port: 8761 spring.security.user.name: admin
spring.security.user.password: medibook123 eureka.client.register-with-eureka: false # server
does not register with itself eureka.client.fetch-registry: false
eureka.server.enable-self-preservation: false # Each microservice: eureka: client:
service-url: defaultZone: http://admin:medibook123@localhost:8761/eureka/ instance:
prefer-ip-address: true
```

Spring Boot Admin Server is integrated in admin-service. It connects to Eureka to automatically discover all registered services and provides a UI dashboard for health monitoring, log levels, metrics, and environment properties.

5.11 Load Balancing

The API Gateway uses **lb://** prefix (Spring Cloud LoadBalancer) with Eureka. When multiple instances of a service are running, requests are distributed round-robin automatically. No Ribbon (deprecated) — uses Spring Cloud LoadBalancer.

```
# api-gateway/application.yml spring.cloud.gateway.routes: - id: appointment-service uri:
lb://appointment-service # lb:// = load balanced via Eureka predicates: -
Path=/appointments/** # Feign client (appointment-service calling schedule-service):
@FeignClient(name = "schedule-service") // service name in Eureka public interface SlotClient
{ @GetMapping("/slots/{slotId}") SlotDto getSlotById(@PathVariable int slotId); }
```

5.12 Bean Lifecycle, IoC, Bean Factory vs ApplicationContext

• IoC (Inversion of Control)

Spring container creates and manages objects (beans). Instead of your code creating dependencies (new AppointmentRepository()), Spring injects them. The developer declares what is needed (@Autowired); Spring decides how/when to create it.

• Bean Lifecycle

1. BeanDefinition loaded (classpath scan finds @Service, @Repository, etc.) 2. BeanFactory instantiates the class 3. Dependency injection (@Autowired fields/constructors filled) 4. @PostConstruct method runs (initialization) 5. Bean is ready to use (used by application) 6. @PreDestroy method runs (cleanup) 7. Bean garbage collected when context closes

• BeanFactory vs ApplicationContext

BeanFactory: Basic IoC container. Lazy initialization. No AOP, no events, no i18n. ApplicationContext: Full-featured container (extends BeanFactory). Eager initialization by default. Supports: AOP proxy creation, @EventListener, @Scheduled, MessageSource, @Transactional. MediBook uses ApplicationContext (via SpringApplication.run()).

• Autowiring Modes

@Autowired by type (default). If multiple beans of same type: @Qualifier("name"). Constructor injection preferred for mandatory dependencies (testability). MediBook uses field injection (@Autowired) for conciseness.

5.13 Java 8+ Features Used in MediBook

```
// 1. Streams (used in ScheduleServiceImpl): List<AvailabilitySlot> slots = IntStream.range(0, count) .mapToObj(i -> AvailabilitySlot.builder() .providerId(providerId) .date(startDate.plusDays(i)) .build()) .collect(Collectors.toList()); // 2. Optional (Repository returns Optional): userRepository.findByEmail(email) .orElseThrow(() -> new ResourceNotFoundException("User", "email", email)); // 3. Lambda (Functional Interface in SecurityConfig): .csrf(csrf -> csrf.disable()) .sessionManagement(session -> session .sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // 4. Method References: slots.stream().filter(Slot::isBooked).collect(Collectors.toList()); // 5. Default Methods in interfaces: default boolean isExpired() { return this.date.isBefore(LocalDate.now()); } // 6. @FunctionalInterface — custom exception suppliers: Supplier<ResourceNotFoundException> notFound = () -> new ResourceNotFoundException("Appointment", "id", id);
```

SECTION 6 — Coding Ability [Score Criterion #6]

6. Coding Ability — Key Implementations

6.1 Frontend: React States, Props, HTTP Calls

The MediBook frontend uses **React 18 + Vite** with React Router v6 for client-side routing, useState/useEffect for state management, and Axios for HTTP calls.

Routing (App.jsx)

```
// Role-based PrivateRoute function PrivateRoute({ children, role }) { const user = getUser();
// from localStorage if (!user) return <Navigate to="/login" replace />; if (role && user.role
!== role) return <Navigate to="/" replace />; return children; } // Usage in Routes: <Route
path="/patient" element={ <PrivateRoute role="Patient"><PatientDashboard /></PrivateRoute> }
/>
```

State & Props Pattern (BookAppointmentPage)

```
// State: const [slots, setSlots] = useState([]); const [selectedSlot, setSelectedSlot] =
useState(null); const [loading, setLoading] = useState(false); // useEffect to fetch slots
when date changes: useEffect(() => { if (!selectedDate) return; setLoading(true);
slotAPI.getAvailable(providerId, selectedDate) .then(res => setSlots(res.data)) .catch(err =>
setError(err.message)) .finally(() => setLoading(false)); }, [selectedDate, providerId]); //
HTTP call to book: const handleBook = async () => { const res = await appointmentAPI.book({
patientId: user.userId, providerId, slotId: selectedSlot.slotId, serviceType,
modeOfConsultation }); // Then initiate payment: const payRes = await paymentAPI.initiate({
appointmentId: res.data.appointmentId, amount }); // Open Razorpay:
openRazorpayPopup(payRes.data.razorpayOrderId); };
```

HTTP Calls (api.js — Axios Interceptors)

```
// Request interceptor: attach JWT api.interceptors.request.use((config) => { const token =
localStorage.getItem("medibook_token"); if (token) config.headers.Authorization = `Bearer
${token}`; return config; }); // Response interceptor: handle 401
api.interceptors.response.use( (res) => res, (err) => { if (err.response?.status === 401) {
localStorage.clear(); window.location.href = "/login"; } return Promise.reject(err); } );
```

Observable vs Promise (HTTP Calls in React)

React uses **Promise-based** HTTP calls (Axios returns Promises). Angular uses Observable (RxJS). Difference:

Feature	Promise (Axios/React)	Observable (Angular)
Execution	Eager — runs immediately	Lazy — runs when subscribed
Values	Single value resolution	Multiple values over time (stream)
Cancellation	AbortController needed	unsubscribe() built-in
JSON parsing	.then(res => res.data)	response.json() or Angular HttpClient auto-parses
Cast to JSON	axios parses JSON automatically; fetch: resp.json() returns Promise<any>	HttpClient auto-deserializes: this.http.get<User>("/user")

6.2 Backend: Booking Flow with Feign Client

```
// SlotClient.java (Feign) @FeignClient(name = "schedule-service") public interface SlotClient
{ @GetMapping("/slots/{slotId}") SlotDto getSlotById(@PathVariable int slotId);
@PutMapping("/slots/{slotId}/book") void bookSlot(@PathVariable int slotId);
@PutMapping("/slots/{slotId}/release") void releaseSlot(@PathVariable int slotId); } //
AppointmentServiceImpl.bookAppointment(): @Transactional public Appointment
bookAppointment(AppointmentRequest request) { SlotDto slot =
slotClient.getSlotById(request.getSlotId()); if (slot.isBooked()) throw new
BadRequestException("Slot already booked"); Appointment appt = Appointment.builder()
.patientId(request.getPatientId()) .slotId(request.getSlotId()) .status("SCHEDULED") .build();
Appointment saved = appointmentRepository.save(appt);
slotClient.bookSlot(request.getSlotId()); // mark slot as booked
eventPublisher.publishBooked(saved); // async RabbitMQ event return saved; }
```

6.3 Scheduled Jobs Implementation

```
// SlotExpiryScheduler.java @Component public class SlotExpiryScheduler { @Autowired private
ScheduleService scheduleService; @Scheduled(cron = "0 0 0 * * *") // every midnight public
void purgeExpiredSlots() { scheduleService.deleteExpiredSlots(); // deletes: date < today AND
isBooked = false } } // NoShowDetectionScheduler.java @Scheduled(cron = "0 0 * * * *") //
every hour public void detectNoShows() { List<Appointment> scheduled =
repo.findByStatus("SCHEDULED"); scheduled.stream() .filter(a ->
a.getAppointmentDate().isBefore(LocalDate.now()) ||
(a.getAppointmentDate().isEqual(LocalDate.now()) && a.getEndTime().isBefore(LocalTime.now())))
.forEach(a -> service.updateStatus(a.getAppointmentId(), "NO_SHOW")); }
```

SECTION 7 — Validation & Exception Handling [Score Criterion #7]

7. Validation: Frontend + Backend & Exception Handling

7.1 Backend Validation (@Valid + Custom Exceptions)

```
// AppointmentRequest.java (DTO): public class AppointmentRequest { @NotNull(message =
"Patient ID is required") private Integer patientId; @NotNull(message = "Slot ID is required")
private Integer slotId; @NotBlank(message = "Service type is required") private String
serviceType; @NotBlank(message = "Consultation mode is required") private String
modeOfConsultation; } // AppointmentResource.java: @PostMapping("/book") public
ResponseEntity<Appointment> book(@Valid @RequestBody AppointmentRequest req) { return
ResponseEntity.status(201).body(service.bookAppointment(req)); }
```

7.2 Global Exception Handler (@RestControllerAdvice)

```
@RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(ResourceNotFoundException.class) public ResponseEntity<ErrorResponse>
handleNotFound(ResourceNotFoundException ex, HttpServletRequest req) { return
ResponseEntity.status(404).body( new ErrorResponse(404, "Not Found", ex.getMessage(),
req.getRequestURI(), LocalDateTime.now())); }
@ExceptionHandler(MethodArgumentNotValidException.class) public
ResponseEntity<Map<String,Object>> handleValidation(MethodArgumentNotValidException ex) {
Map<String,String> errors = new HashMap<>(); ex.getBindingResult().getFieldErrors()
.forEach(fe -> errors.put(fe.getField(), fe.getDefaultMessage())); return
ResponseEntity.badRequest().body(Map.of("errors", errors, "status", 400, "error", "Validation
Failed")); } @ExceptionHandler(ForbiddenException.class) // 403
@ExceptionHandler(UnauthorizedException.class) // 401
@ExceptionHandler(BadRequestException.class) // 400
@ExceptionHandler(DuplicateResourceException.class) // 409 @ExceptionHandler(Exception.class)
// 500 fallback }
```

7.3 Custom Exception Classes

Exception	HTTP Status	When Used
ResourceNotFoundException	404 Not Found	Entity not found by ID or criteria
BadRequestException	400 Bad Request	Business rule violation (slot already booked, etc.)
DuplicateResourceException	409 Conflict	Unique constraint violated (e.g., email exists)
UnauthorizedException	401 Unauthorized	Missing or invalid JWT token
ForbiddenException	403 Forbidden	Valid token but insufficient permissions

7.4 Frontend Validation

```
// In RegisterPage.jsx / LoginPage.jsx: const [errors, setErrors] = useState({}); const
validate = () => { const errs = {}; if (!form.email.includes("@")) errs.email = "Invalid
email"; if (form.password.length < 6) errs.password = "Min 6 characters"; if
(!form.fullName.trim()) errs.fullName = "Name is required"; setErrors(errs); return
Object.keys(errs).length === 0; }; const handleSubmit = async () => { if (!validate()) return;
try { await authAPI.register(form); } catch (err) { setErrors({ api:
err.response?.data?.message }); } };
```

SECTION 8 — AOP & Logging [Score Criterion #8]

8. AOP: Logger Using Aspect Class

AOP (Aspect-Oriented Programming) separates cross-cutting concerns (logging, security, transactions) from business logic. Spring uses proxy-based AOP with AspectJ annotations.

8.1 Service Layer Logger Aspect

```
@Aspect @Component public class ServiceLoggingAspect { private static final Logger log = LoggerFactory.getLogger(ServiceLoggingAspect.class); // Pointcut: all methods in any service implementation @Pointcut("execution(* com.medibook..service.impl.*.*(..))") public void serviceMethods() {} @Around("serviceMethods()") public Object logServiceCall(ProceedingJoinPoint pjp) throws Throwable { String method = pjp.getSignature().toShortString(); log.info("[ENTER] {} args={}", method, Arrays.toString(pjp.getArgs())); long start = System.currentTimeMillis(); try { Object result = pjp.proceed(); long duration = System.currentTimeMillis() - start; log.info("[EXIT] {} duration={}ms", method, duration); return result; } catch (Exception e) { log.error("[ERROR] {} threw: {}", method, e.getMessage()); throw e; } } @AfterThrowing(pointcut = "serviceMethods()", throwing = "ex") public void logException(JoinPoint jp, Exception ex) { log.error("[EXCEPTION] in {} → {}", jp.getSignature(), ex.getMessage()); } }
```

8.2 Log4J / SLF4J Configuration

```
# application.yml — redirect logs to file logging: level: com.medibook: INFO org.springframework.cloud.gateway: INFO file: name: logs/medibook-appointment.log pattern: file: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n" console: "%d{HH:mm:ss} %-5level %logger{20} - %msg%n"
```

8.3 AOP Concepts Summary

AOP Concept	Annotation	Meaning
Aspect	@Aspect @Component	The class containing cross-cutting logic
Pointcut	@Pointcut("execution(*...)")	Which methods to intercept
Before Advice	@Before	Runs before the method executes
After Returning	@AfterReturning	Runs after method returns successfully
After Throwing	@AfterThrowing	Runs if method throws exception
Around Advice	@Around	Wraps the method; can modify args/return/throw
Join Point	ProceedingJoinPoint	The actual method being intercepted

SECTION 9 — Swagger UI [Score Criterion #9]

9. Swagger UI (OpenAPI 3.0)

MediBook uses **SpringDoc OpenAPI 3.0** (springdoc-openapi-starter-webmvc-ui) to auto-generate interactive API documentation from code annotations.

9.1 Dependencies

```
<!-- pom.xml --> <dependency> <groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId> <version>2.5.0</version>
</dependency>
```

9.2 OpenAPI Configuration

```
@Configuration public class OpenApiConfig { @Bean public OpenAPI mediBookOpenAPI() { return
new OpenAPI() .info(new Info() .title("MediBook Payment Service API") .description("Payment
processing, refunds, and earnings") .version("1.0.0") .contact(new Contact().name("MediBook
Team"))) .addSecurityItem(new SecurityRequirement().addList("Bearer Auth")) .components(new
Components().addSecuritySchemes( "Bearer Auth", new SecurityScheme()
.type(SecurityScheme.Type.HTTP) .scheme("bearer") .bearerFormat("JWT"))); } }
```

9.3 Controller Annotations

```
@RestController @Tag(name = "Appointment", description = "Appointment booking lifecycle")
@RequestMapping("/appointments") public class AppointmentResource { @Operation(summary = "Book
a new appointment", description = "Patient books a slot; marks slot as booked; fires RabbitMQ
event") @ApiResponses({ @ApiResponse(responseCode = "201", description = "Appointment
created"), @ApiResponse(responseCode = "400", description = "Slot already booked"),
@ApiResponse(responseCode = "401", description = "Unauthorized") }) @PostMapping("/book")
public ResponseEntity<Appointment> book(@Valid @RequestBody AppointmentRequest req) { return
ResponseEntity.status(201).body(service.bookAppointment(req)); } }
```

9.4 Swagger UI Endpoints per Service

Service	Swagger UI URL	API Docs URL
auth-service	http://localhost:8081/swagger-ui.html	http://localhost:8081/v3/api-docs
appointment-service	http://localhost:8084/swagger-ui.html	http://localhost:8084/v3/api-docs
payment-service	http://localhost:8085/swagger-ui.html	http://localhost:8085/v3/api-docs
schedule-service	http://localhost:8083/swagger-ui.html	http://localhost:8083/v3/api-docs
notification-service	http://localhost:8087/swagger-ui.html	http://localhost:8087/v3/api-docs

Swagger UI is permitted without authentication in SecurityConfig: .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()

SECTION 10 — SonarQube & Test Coverage [Score Criterion #10]

10. SonarQube Report & Test Coverage

10.1 Sample JUnit Test — PaymentServiceImpl

```
// PaymentServiceImplTest.java @ExtendWith(MockitoExtension.class) class
PaymentServiceImplTest { @Mock private PaymentRepository paymentRepository; @Mock private
AppointmentClient appointmentClient; @InjectMocks private PaymentServiceImpl paymentService;
@Test void testInitiatePayment_Success() { AppointmentDto appt = new AppointmentDto();
appt.setAppointmentId(1); when(appointmentClient.getById(1)).thenReturn(appt);
when(paymentRepository.findByAppointmentId(1)).thenReturn(Optional.empty());
when(paymentRepository.save(any())).thenAnswer(inv -> inv.getArgument(0)); PaymentRequest req
= PaymentRequest.builder() .appointmentId(1).patientId(101).amount(500.0)
.paymentMethod("UPI").build(); PaymentResponse resp = paymentService.initiatePayment(req);
assertNotNull(resp); assertEquals("PENDING", resp.getStatus());
verify(paymentRepository).save(any(Payment.class)); } }
```

10.2 SonarQube Setup

```
# pom.xml — add SonarQube plugin: <plugin> <groupId>org.sonarsource.scanner.maven</groupId>
<artifactId>sonar-maven-plugin</artifactId> <version>3.11.0.3922</version> </plugin> # JaCoCo
for coverage: <plugin> <groupId>org.jacoco</groupId>
<artifactId>jacoco-maven-plugin</artifactId> <version>0.8.11</version> <executions>
<execution><goals><goal>prepare-agent</goal></goals></execution>
<execution><id>report</id><phase>test</phase> <goals><goal>report</goal></goals></execution>
</executions> </plugin> # Run analysis: mvn clean verify sonar:sonar \
-Dsonar.projectKey=medibook-appointment-service \ -Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=your_token
```

10.3 SonarQube Metrics to Target

Metric	Target	MediBook Status
Code Coverage	≥ 80%	PaymentServiceImpl has JUnit tests; coverage tracked via JaCoCo
Bugs	0 Critical/Blocker	GlobalExceptionHandler prevents uncaught exceptions
Vulnerabilities	0 Critical	No SQL injection (JPA); passwords BCrypt; JWT validated
Code Smells	< 20 Minor	@Builder, consistent naming, no magic numbers
Duplications	< 10%	Shared DTO patterns; no copy-paste service logic
Maintainability Rating	A	Service/Interface separation; single responsibility
Security Rating	A	BCrypt + JWT + HTTPS; CSRF disabled for correct reason

SECTION 11 — Circuit Breaker & Resilience4J [Score Criterion #11]

11. Circuit Breaker & Resilience4J

If schedule-service is down, appointment-service must not crash. **Resilience4J** provides a Circuit Breaker that detects failures and falls back gracefully.

11.1 Circuit Breaker States

State	Behavior	Transition
CLOSED	Normal operation; all requests pass through	→ OPEN when failure rate > threshold
OPEN	Requests fail fast with fallback; no calls to service	→ HALF_OPEN after waitDuration
HALF_OPEN	Limited requests probe if service recovered	→ CLOSED if success; OPEN if fail

11.2 Dependency Setup

```
<!-- pom.xml --> <dependency> <groupId>io.github.resilience4j</groupId>
<artifactId>resilience4j-spring-boot3</artifactId> <version>2.2.0</version> </dependency>
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-aop</artifactId> </dependency>
```

11.3 Configuration

```
# application.yml (appointment-service) resilience4j: circuitbreaker: instances: slotService:
slidingWindowSize: 10 failureRateThreshold: 50 waitDurationInOpenState: 30s
permittedNumberOfCallsInHalfOpenState: 3 automaticTransitionFromOpenToHalfOpenEnabled: true
retry: instances: slotService: maxAttempts: 3 waitDuration: 500ms
```

11.4 Implementation on Feign Client

```
// AppointmentServiceImpl.java @CircuitBreaker(name = "slotService", fallbackMethod =
"bookAppointmentFallback") @Retry(name = "slotService") public Appointment
bookAppointment(AppointmentRequest request) { SlotDto slot =
slotClient.getSlotById(request.getSlotId()); // ... rest of booking logic } // Fallback method
(same signature + Throwable param): public Appointment
bookAppointmentFallback(AppointmentRequest request, Throwable ex) {
log.error("schedule-service is DOWN. Cannot book appointment: {}", ex.getMessage()); throw new
ServiceUnavailableException( "Booking service temporarily unavailable. Please try again in a
few minutes."); } // Health indicator exposed at /actuator/health: // { "circuitBreakers": {
"slotService": { "state": "CLOSED" } } }
```

11.5 Why Resilience4J over Hystrix?

Hystrix is in maintenance mode (Netflix OSS). Resilience4J is the recommended replacement for Spring Boot 3.x. Key advantages: lighter dependency footprint, functional programming style, better integration with Micrometer metrics, and support for reactive programming.

11.6 Additional Resilience Patterns

Pattern	Annotation	Purpose
Circuit Breaker	@CircuitBreaker	Stop cascading failures when service is down

Pattern	Annotation	Purpose
Retry	@Retry	Auto-retry transient failures (network blip)
Rate Limiter	@RateLimiter	Prevent overwhelming a downstream service
Bulkhead	@Bulkhead	Isolate service calls; prevent thread pool exhaustion
TimeLimiter	@TimeLimiter	Timeout slow calls; prevent hanging requests
Fallback (Gateway)	FallbackController	Return cached response if service is unreachable